

The Threat Hunting Playbook

A complete, detailed guide to proactive threat hunting —
from a hypothesis to a documented, repeatable hunt

*From a single hunch about attacker behavior to a documented,
repeatable hunt procedure, this playbook builds the hypothesis-
driven methodology that turns raw log access into proactive detection.*

Author: Swastik Sagar
Final-year Computer Science Undergraduate
SOC Analyst Intern

August 22, 2026

Preface

Incident response starts when an alert fires; threat hunting starts before that, on the assumption that something is already present that no alert has caught yet. It's a fundamentally different posture — not waiting for a detection rule to trigger, but actively going looking for the behaviors that rules haven't been written for, or that were deliberately built to evade the rules that exist.

This playbook treats hunting as a discipline built on hypotheses, not a vague activity of "looking around in the SIEM." Each chapter builds toward that: the maturity model that describes where a hunting program actually sits, hypothesis construction grounded in MITRE ATT&CK, the data sources a hypothesis needs to be testable at all, and then specific hunts for the technique categories that matter most — persistence, lateral movement, C2, and exfiltration — before covering how a successful ad hoc hunt becomes a repeatable, documented, eventually automated part of the detection program.

What this book covers

- The threat hunting mindset and the hunting maturity model
- Hypothesis-driven hunting, step by step
- Data sources: what a hunt actually needs to be possible
- Using MITRE ATT&CK as a hunting framework
- Hunting for persistence mechanisms
- Hunting for lateral movement
- Hunting for command and control
- Hunting for data exfiltration
- Building hunt queries in a SIEM
- Documenting and operationalizing successful hunts
- Measuring whether a hunting program is actually working

Swastik Sagar

Contents

Preface	i
1 The Threat Hunting Mindset	1
1.1 Hunting vs. monitoring	1
1.2 The hunting maturity model	1
2 Hypothesis-Driven Hunting	2
2.1 The hunting cycle	2
2.2 Where hypotheses come from	2
2.3 Writing a good hypothesis	2
3 Data Sources for Hunting	3
3.1 What a hunt actually needs	3
3.2 Data retention and hunting	3
4 MITRE ATT&CK as a Hunting Framework	4
4.1 Why ATT&CK fits hunting so well	4
4.2 Using the ATT&CK matrix to plan hunts	4
5 Hunting for Persistence	5
5.1 Why persistence is a high-value hunt target	5
5.2 Common persistence locations	5
6 Hunting for Lateral Movement	6
6.1 The signature of lateral movement	6
6.2 What to hunt for	6
7 Hunting for Command and Control	7
7.1 What C2 traffic tends to look like	7
7.2 Beacons detection approach	7
8 Hunting for Data Exfiltration	8
8.1 What to look for	8

9	Building Hunt Queries	9
9.1	From hypothesis to query, step by step	9
9.2	A general hunt query pattern	9
10	Documenting and Operationalizing Hunts	10
10.1	Why documentation matters as much as the hunt itself	10
10.2	What to document	10
10.3	From hunt to standing detection	10
11	Measuring Hunt Program Effectiveness	11
11.1	Metrics worth tracking	11
11.2	Reporting hunt program value upward	11

The Threat Hunting Mindset

1.1 Hunting vs. monitoring

	Approach
Monitoring	Reactive; waits for a detection rule to fire, then investigates
Hunting	Proactive; assumes detection gaps exist and actively searches for evidence of compromise

Hunting starts from an assumption of failure

The working assumption behind every hunt is that some compromise already exists and simply hasn't triggered an alert — either because no rule covers that specific behavior, or because the activity was deliberately designed to blend in with normal traffic. That assumption is what justifies looking without a starting alert at all.

1.2 The hunting maturity model

Level	Characteristics
0	Relies entirely on automated alerting, no hunting capability
1	Incorporates threat intelligence into searches, still largely reactive
2	Follows procedures created by others, hunts on a defined cadence
3	Creates new hunting procedures, hypothesis-driven
4	Automates successful hunts into standing detections, closing the loop

Level 4 is the real goal, not level 3

A hunting program that never automates its own successful hunts is doing the same manual work repeatedly instead of compounding it into better standing detection — the maturity model's real endpoint is a program where good hunts become new detection rules, not more hunts.

Hypothesis-Driven Hunting

2.1 The hunting cycle

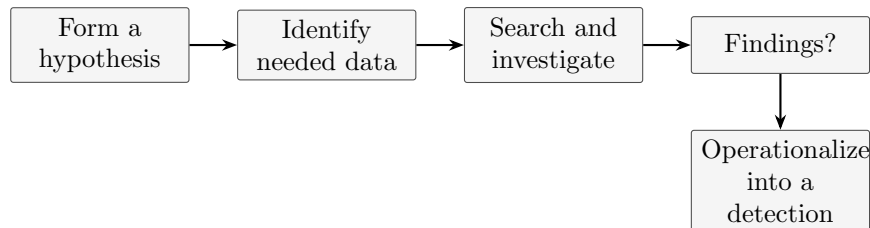


Figure 2.1. *A hunt with no findings is not a wasted hunt – ruling out a hypothesis is itself useful information about coverage.*

2.2 Where hypotheses come from

- MITRE ATT&CK techniques not yet covered by existing detections
- Threat intelligence describing tactics used by adversaries relevant to the organization's sector
- Findings from a recent incident, extended into a broader hunt across the environment
- Anomalies noticed during unrelated analysis, worth a dedicated look

2.3 Writing a good hypothesis

A well-formed hunting hypothesis

If an attacker has established persistence via a scheduled task, then there should be evidence of an unusual scheduled task creation event (Event ID 4698) on one or more hosts, created by a process or account that doesn't normally perform this action.

A hypothesis needs to be falsifiable

"Something suspicious might be happening" isn't a hypothesis — it's a feeling. A useful hunting hypothesis specifies exactly what evidence would confirm it and, just as importantly, what searching and finding nothing would mean about the environment's actual state.

Data Sources for Hunting

3.1 What a hunt actually needs

A hypothesis is only testable if the data it depends on is actually being collected — confirming data availability is a necessary first step before investing time in a hunt, not an afterthought if the hunt comes up empty.

Source	Typical hunting value
Endpoint process logs (Sysmon, EDR)	Process creation, parent-child relationships, command lines
Authentication logs	Logon patterns, privilege use, account anomalies
Network flow / DNS logs	Beaconing patterns, unusual destinations, data volume
Cloud audit logs (CloudTrail and similar)	API-level activity for cloud-hosted infrastructure

A gap here is itself a finding

Discovering that a hypothesis can't be tested because the needed log source doesn't exist, or doesn't retain data long enough, is a valid and valuable hunt outcome — it identifies a detection gap just as concretely as finding actual malicious activity would.

3.2 Data retention and hunting

Hunts often look back further than a typical investigation — a sophisticated persistence mechanism might have been sitting undetected for months. Retention policy directly bounds how far back a hunt can actually look, which makes retention a hunting-program decision, not just a storage-cost one.

MITRE ATT&CK as a Hunting Framework

4.1 Why ATT&CK fits hunting so well

ATT&CK catalogs adversary behavior as tactics and techniques rather than specific tools or indicators — exactly the level of abstraction a hunting hypothesis needs, since it targets *how* an attacker operates rather than any single, easily-changed artifact.

4.2 Using the ATT&CK matrix to plan hunts

- Cross-reference existing detection coverage against the full matrix to find genuine gaps
- Prioritize techniques relevant to the organization's actual threat model, not the matrix exhaustively
- Use sub-techniques to scope a hunt precisely rather than an entire broad tactic at once

Example: scoping from a tactic down to a hunt

Tactic: Persistence
Technique: T1053 - Scheduled Task/Job
Sub-technique: T1053.005 - Scheduled Task (Windows)
Hunt: unusual schtasks.exe / Event ID 4698 activity

ATT&CK Navigator for coverage visualization

The ATT&CK Navigator tool lets a hunting program visually track which techniques have detection coverage, which have been hunted, and which remain uncovered — turning coverage tracking into something the whole team can see at a glance rather than something living only in one analyst's head.

Hunting for Persistence

5.1 Why persistence is a high-value hunt target

An attacker who has already achieved persistence has, by definition, survived past initial detection — making persistence mechanisms one of the highest-value places to hunt specifically because their presence implies an existing detection gap already happened once.

5.2 Common persistence locations

Mechanism	What to hunt for
Scheduled tasks / cron	New task creation events from unusual accounts or with unusual command lines
Registry Run keys	New or modified Run/RunOnce entries pointing to uncommon paths
Services	New service installation events (7045), especially binaries outside normal paths
Startup folder items	New files appearing in user or common startup locations

A hunt query sketch, SPL-style

```
index=endpoint EventCode=4698
| where NOT match(TaskContent, "known-good-pattern")
| stats count by host, Author, TaskName
```

Baseline before you hunt

Scheduled tasks and Run keys are also used constantly for entirely legitimate purposes — a hunt here needs a baseline of what's normal for the environment first, or the result is an unmanageable volume of false positives rather than a useful finding.

Hunting for Lateral Movement

6.1 The signature of lateral movement

Lateral movement almost always involves authentication from one internal host to another, often via a small set of well-known administrative protocols — the hunt is less about any single event and more about identifying patterns that don't match how an account or host normally behaves.

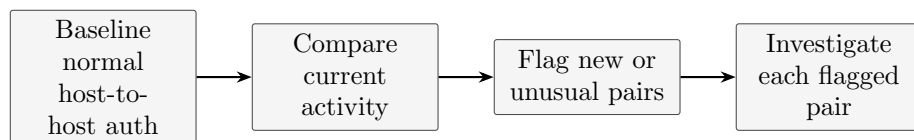


Figure 6.1. *Lateral movement hunting is fundamentally a baseline comparison problem, not a signature-matching one.*

6.2 What to hunt for

- Workstation-to-workstation authentication, which is rare in most environments compared to workstation-to-server
- A single account authenticating to an unusually high number of distinct hosts in a short window
- Use of administrative tools (PsExec, WMI, WinRM) from a host that doesn't normally use them
- Logon Type 3 (network) or Type 10 (RDP) activity outside an account's normal pattern

A hunt query sketch: rare host-to-host pairs

```
index=auth Logon_Type=3
| stats dc(dest_host) as unique_dests by user
| where unique_dests >10
```

Hunting for Command and Control

7.1 What C2 traffic tends to look like

Command and control traffic has to balance stealth against reliability — most C2 frameworks default to periodic check-ins, which produces patterns that stand out once you're specifically looking for regularity rather than volume.

7.2 Beaconsing detection approach

- Group outbound connections by destination and look for highly regular time intervals between connections
- Look for consistent, small payload sizes across many connections to the same destination
- Cross-reference destinations against threat intelligence feeds, but don't rely on this alone — new infrastructure won't be listed yet
- Investigate newly registered or algorithmically-generated-looking domains in DNS logs

A beaconsing hunt query sketch

```
index=network
| bin _time span=1h
| stats count, stdev(_time) as interval_variance by src_ip, dest_ip
| where count >20 AND interval_variance <5
```

Low variance in connection timing, combined with a meaningful connection count, is the core beaconsing signature this query approximates — highly regular intervals are unusual for legitimate application traffic.

Jittered C2 complicates simple interval checks

Modern C2 frameworks often add deliberate jitter to check-in timing specifically to defeat simple regularity-based detection — a mature hunt also looks at longer-window statistical patterns and destination reputation together, not interval regularity alone.

Hunting for Data Exfiltration

8.1 What to look for

- Large, sustained outbound transfers to external or previously-unseen destinations
- Data staged into an archive (zip, rar) shortly before an unusual outbound transfer
- Transfers to cloud storage or file-sharing services outside normal business use
- Activity timed outside a host or account's normal working-hours pattern

A staging-then-exfil hunt query sketch

```
index=endpoint (process=*zip* OR process=*rar*) OR  
index=network bytes_out >500000000  
| stats values(process) as staging_activity,  
sum(bytes_out) as total_out by host, _time
```

Volume alone is a weak signal

A single large transfer isn't inherently suspicious — legitimate backup jobs and data syncs also produce large, regular transfers. The distinguishing signal is usually the destination and surrounding context (staging activity, off-hours timing, an account that doesn't normally move data at all), not size in isolation.

Building Hunt Queries

9.1 From hypothesis to query, step by step

- Start broad enough to avoid missing variations of the behavior, then narrow based on results
- Filter out known-good patterns explicitly, rather than trying to match only known-bad ones
- Aggregate before you enumerate — a stats-based overview surfaces outliers faster than scrolling raw events
- Iterate: a hunt query is rarely right on the first attempt, and that's expected

9.2 A general hunt query pattern

The shape most hunt queries follow

```
index=<relevant source> <broad initial filter>
| stats count by <grouping field(s)>
| where count <comparison> <threshold>
| sort -count
```

This same shape appears across nearly every hunt in this book: filter broadly, aggregate by the field that would reveal the pattern, then apply a threshold that separates normal volume from anomalous volume.

Thresholds need environment-specific tuning

A threshold that works well in one environment can be completely wrong in another — an organization with heavy legitimate automation will have naturally higher baseline counts than one without, which is why every threshold in this book's example queries should be treated as a starting point, not a final answer.

Documenting and Operationalizing Hunts

10.1 Why documentation matters as much as the hunt itself

An undocumented hunt, however successful, has to be reinvented from scratch the next time someone wants to run it — documentation is what turns a one-time investigation into a repeatable capability the whole team can use.

10.2 What to document

Section	Contents
Hypothesis	The specific, falsifiable statement the hunt was testing
Data sources required	What logs/tools the hunt depends on
Query/procedure	The exact steps or query used, ready to re-run
Findings	What was found, including "nothing found" as a valid result
Follow-up actions	Whether this became a new detection, and any remaining gaps identified

10.3 From hunt to standing detection

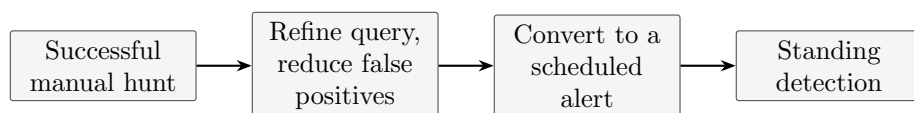


Figure 10.1. *This is the maturity-model level 4 transition from chapter 1 – a hunt that never makes this trip has to be repeated by hand indefinitely.*

Measuring Hunt Program Effectiveness

11.1 Metrics worth tracking

Metric	What it indicates
Hunts conducted per period	Program activity level and consistency
ATT&CK technique coverage	How much of the relevant matrix has been actively hunted, not just theoretically covered
Hunts converted to detections	Whether the program is compounding value over time (maturity level 4)
Mean time to detect, before/after	Whether hunting is measurably improving detection speed for the technique categories hunted

Findings count is a misleading primary metric

A hunting program that measures success purely by "threats found" creates a perverse incentive to find something, anything, to justify its existence — a hunt that rigorously confirms an environment is clean for a given technique is just as valuable as one that finds an active compromise, and a good metrics framework reflects that.

11.2 Reporting hunt program value upward

- Frame findings in terms of risk reduction and coverage improvement, not just incident counts
- Track and report on detection gaps closed, not only compromises found
- Tie hunt outcomes back to the organization's actual threat model, showing relevance rather than just activity volume

The real deliverable is better standing detection

A mature hunting program's most important output isn't any single hunt's findings — it's a security program that detects more, faster, over time, because each hunt either closes a gap directly or confirms one doesn't exist. That compounding effect is what justifies the program's existence far more than any individual hunt's result.